



COS 132 - Imperative Programming

Study Unit 3: Types, Variables and Input

Copyright ©2024 by Linda Marshall, Inge Odendaal and Mark Botros. All rights reserved.

Contents

3.1	Introduction	2
3.2	Primitive Types	2
3.2.1	Ordinal	2
3.2.2	Variable-length	2
3.2.3	Type modifiers	3
3.2.4	Summary of Types	3
3.3	Variables and Constants	4
3.3.1	Variables	4
3.3.2	Constant	4
3.3.2.1	Preprocessor method	5
3.3.2.2	const keyword method	5
3.3.3	Scope	6
3.3.4	Naming conventions	6
3.3.5	Type conversions	7
3.3.6	Outputting variables	8
3.3.7	Advanced Formatting	9
3.4	Operators	10
3.4.1	Assignment	11
3.4.2	Mathematical	12
3.4.2.1	Compound Assignment Operators	13
3.4.3	Comparison operators	13
3.4.4	Boolean Expressions	15
3.4.4.1	Short-Circuiting Operators	17
3.4.5	Operator Precedence	18
3.5	Input	19
3.6	Document Change Log	22

3.1 Introduction

In Study Unit 2, we considered statements and how statements are specified in a program and form the instructions/actions that are carried out by the computer when the program is run.

In this study unit, the focus moves to considering expressions. Expressions are used in statements and conditions. Expressions are any section of code that evaluates to a value. In order to manage values, we need to consider types and how they can be manipulated.

A type provides the blue-print to which a value must adhere. To effectively use the types, we will define variables and constants that we will create within our program. Types in C++ are categorised as Primitive (or built-in), Derived and Abstract (or user-defined). In this study unit the focus will be on the most common Primitive types that relate to mathematical and logic expressions. The basics of strings will also be presented.

The sections which follow, introduce you to the most common primitive types defined in C++ and the operators that can be applied with variables and constants of these types. Variables and constants, together with operators are used to build expressions which are in turn included in statements.

3.2 Primitive Types

Primitive types fall into two main categories. Types where the previous and next values of the type are predefined and types where the previous and next values could respectively be any value smaller or greater than the current value. Values that are predefined and therefore predictable are said to be ordinal, while values where we cannot determine the next value in the sequence are values of variable-length types. Each of these types will be discussed in the sections that follow.

3.2.1 Ordinal

Ordinal types are Integer, Character and Boolean. The keywords used to represent these types are `int`, `char` and `bool` respectively.

3.2.2 Variable-length

Variable-length types include floating points, of which there are two types: `float` and `double`. The difference between the two lies in their precision. Floating point data type stores single-precision floating-point values, with `float` typically requiring 4 bytes of memory space. Double floating point, on the other hand, stores double-precision floating-point values and typically requires 8 bytes of memory space.

The `string` type is defined in a library and is not a bona-fide primitive type as it comprises a sequence of primitive `char` types. A variable of type `char` represents a single character, for example 'a'. A variable of type `char*` is the representation used to define a `string`. We will consider the details of the `string` type and how it is defined in a later study unit. In this study unit, we will be reading strings in from the keyboard and writing strings out to the screen.

3.2.3 Type modifiers

In C++ type modifiers are used to change the properties of basic data types, so that programmers can define variables more suited to the specific application.

These modifiers include:

- `signed` (for `int` and `char`)
- `unsigned` (for `int` and `char`)
- `short` (for `int`)
- `long` (for `int` and `double`)

Each of these modifiers serve a different purpose in modifying the storage size and range of values that each data type can represent.

- **Signed:** Can store positive and negative numbers. It does not change the storage size, but halves the positive range to accommodate negative values. The default for C++ integers.
- **Unsigned:** Can only store positive numbers and zero, doubling the positive range by eliminating the need for bits to represent negative values.
- **Short:** Reduces storage size and range of values. Less memory usage, but also less range.
- **Long:** Increases storage size and range of values. More memory usage, but also a greater range.

Modifiers can be combined for more precise control. The next section provides an overview of the basic types and illustrates what effect the modifiers has on the basic types. Note, the modified types are not exhaustive.

3.2.4 Summary of Types

Table 3.1 provides an overview of the C++ types. The table breaks the types into their high level classifications, the type name used in C++, the range of values represented by the type and the number of bytes used to store the type in memory.

The type presented in the table form the basis of the types in C++. They are used to define variables and constants in Section 3.3 and will in a later study unit be used to define parameter and return value types when we define functions.

Type	C++ type name	Byte size	Range
Ordinal	int	4	-2147483648 $\rightarrow$ 2147483647
	char	1	-128 $\rightarrow$ 127
	bool	1	true or false
Variable-length	float	4	$-3.4 \times 10^{38} \rightarrow 3.4 \times 10^{38}$
	double	8	$-1.7 \times 10^{308} \rightarrow 1.7 \times 10^{308}$
Variable-length (Library-based)	std::string	24	
Modifiers	short int	2	-32768 $\rightarrow$ 32767
	unsigned short int	2	0 $\rightarrow$ 65535
	unsigned long int	4	0 $\rightarrow$ 4294967295
	long long int	8	$-2^{63} \rightarrow 2^{63}-1$
	long double	12	$-1.1 \times 10^{4932} \rightarrow 1.1 \times 10^{4932}$

Table 3.1: Basic and Modified Types

3.3 Variables and Constants

Defining types and assigning values to the types allow these values to be used in expressions. This section will present two ways of defining these values. The first is to define a variable and the second a constant.

3.3.1 Variables

A variable is a letter or word that can stand in for any number, character, string, list, *etc.*. Think of a variable as a place-holder for a value. The value in the place-holder can change during the course of the execution of a program. A variable is used in an expression and may result in different results when the expression is evaluated due to the value associated to the variable varying.

Variables in C++ are defined by providing a type and name for the variable in the format:

<typename> <variablename>

where **typename** is one of the types given in Table 3.1 and **variablename** the name the variable will be referred to by the program.

Once a variable has been defined, a value can be assigned to the variable (refer to Section 3.4.1). The variable is then a representation for that value until the value changes. An example of a definition of a variable is given by:

```
int radius;
```

In this example, a variable with the name **radius** is defined of type **int**. This means **radius** may take on values that are integers.

3.3.2 Constant

Constants are used as a naming mechanism for values that may not change during the life of a program. Typical values that can be defined as a constant are **PI**, **MAX_STUDENTS**,

etc. These values are defined once and can be used a number of times within the program. There are two ways that a constant can be defined in C++. The first is by using a preprocessor directive and the second is by making use of the `const` keyword.

3.3.2.1 Preprocessor method

A constant defined using the preprocessor method is defined by using the keyword `define`, providing a name for the constant and a corresponding value. The syntax for defining a constant using the preprocessor method is given by:

```
#define <CONSTANT_NAME> <constant_value>
```

As the constant is defined outside the main function (and later you will see, outside any other function), it is available to all parts of the program. Below is an example to define PI.

```
#define PI 3.1415
```

Program listing 3.1 illustrates where the constant is defined and how it is used in the main function.

```
#include <iostream>
```

```
#define PI 3.1415
```

```
using namespace std;
```

```
int main() {  
    cout << "The area of a circle with a radius of 10cm is : "  
    << PI * 10 * 10;  
  
    return 0;  
}
```

Listing 3.1: Defining a constant with the preprocessor method

3.3.2.2 const keyword method

Defining a constant using the keyword method is similar to defining a variable in C++. To define a constant the keyword `const` must be specified followed by the definition of constant. A constant is identified by a name and a type. The syntax to define the constant is given by:

```
const <typename> <CONSTANT_NAME> = <constant_value>;
```

The value the name of the constant represents, must be assigned to the constant on creation. For example, the constant value 10 is named `RADIUS` and is of type `int` in the example below. Rather than using 10 in the code when referring to the radius.

```
const int RADIUS = 10;
```

Depending on where a constant is defined in a C++ program, will determine the scope of the constant. That is from where the constant is accessible. Refer to Section 3.3.3 for a more detailed discussion.

```
1 #include <iostream>
2
3 const float PI = 3.1415;
4
5 using namespace std;
6
7 int main() {
8     const int RADIUS = 10;
9     cout << "The area of a circle with a radius of 10cm is : "
10         << PI * RADIUS * RADIUS;
11
12     return 0;
13 }
```

Listing 3.2: Keyword `const` method

The types of the values must be valid C++ value types. These types are the same as the types used to define variables.

Question 3.1

What is the main difference between a constant and a variable?

3.3.3 Scope

Defining the constant outside the main function enables the constant to be used globally. If the constant is defined within the main function, only the main function knows about the constant and can use the constant.

3.3.4 Naming conventions

It is important that when you choose a name for a variable or constant, it is descriptive. This will make it easier to read the code. Code that is easier to follow is easier to maintain. The program code will also not be lost within comments trying to explain the code if the code itself can be self explanatory.

The variable name must begin with a letter and may not contain spaces. You will see that throughout this text, variables start with small letters and when a variable consists of more than one word, the other words start with capital letters¹. This is merely a convention we follow for the sake of readability, for example `drawDoubleStoryHouse` is more readable than `drawdoublestoryhouse`.

Constant names are typically written in all capital letters to easily distinguish them from variables.

The variable type and the keyword constant type is any valid type defined by the C++ language or the programmer. Refer to Table 3.1. The preprocessor constant type is determined by the compiler based on the value given to the constant.

¹CamelCase

3.3.5 Type conversions

Type conversions occurs when the variables defined data type is converted into another data type. This is often needed when performing operations on values of two different types, when passing a value as a parameter, and when assigning a value to a variable. Type conversions can occur automatically (known as **implicit conversions**) or can be performed manually by the programmer (known as **explicit conversions** or **typecasts**)

1. Implicit Conversions

These conversions are performed automatically by the compiler when multiple compatible types are encountered in an expression. A compiler can only do these conversions when it considers them safe. An example of implicit type conversion:

```
int x = 10;
double y = x; // converting int to double
```

2. Explicit Conversions

Explicit conversions are not done by the compiler, instead you must manually add these conversions in your program. These are also known as **typecasts**, since you are explicitly casting one variable type to another.

It is important to be aware of type conversions while writing C++ code. It is also important to note when the conversions occur. An error, commonly known as a **integer division truncation error**, often happens due to a misunderstanding of type conversions.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a = 5, b = 2;
6     float result;
7
8     result = a / b; // Both a and b are integers
9     cout << "Result:_" << result << endl; // Expected: 2.5, Actual output:
10    return 0;
11 }
```

Listing 3.3: Integer Division Illustrating a Truncation Error

In Listing 3.3 the following is happening:

1. The operation a / b is performed using integer arithmetic because both operands are integers (thus no type conversion is necessary). The result is 2 instead of 2.5, truncating the decimal part.
2. This integer result is then implicitly converted to a float when it's assigned to result. However, the precision loss (from truncation) has already occurred, so the conversion to float changes the type but is unable to restore the lost fractional part.

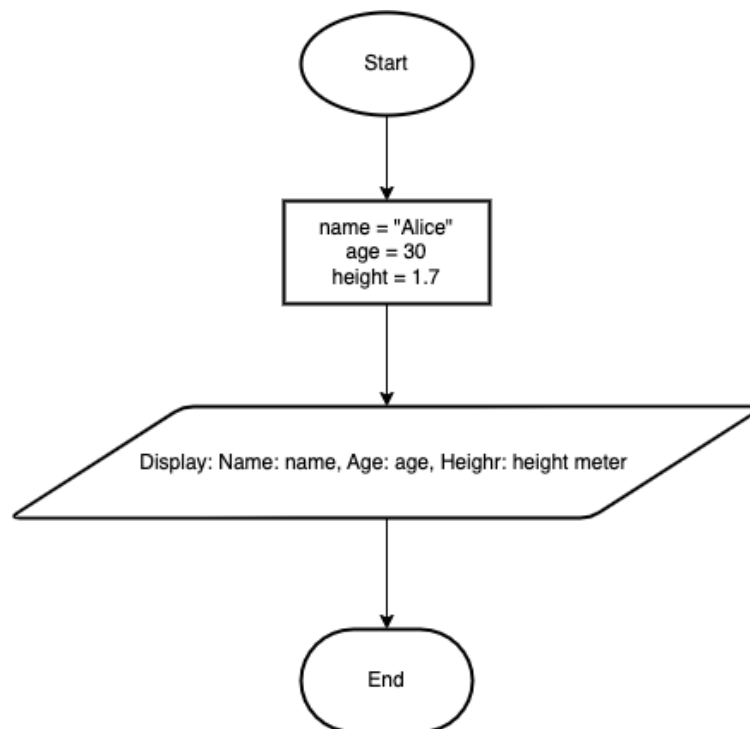
In order to perform floating point arithmetic, one of the operands must be a floating point.

3.3.6 Outputting variables

The << operator is used, not only to output string literals to the console, but also for chaining multiple outputs together, including variables/constants of various data types. Therefore, you can build complex structures/sentences that include both text and variable/constant values

You can chain the << operator repeatedly in a single statement to output different values (text literals or variables/constants) consecutively. Thus you can combine text with variables without multiple cout operations.

Consider the following flowchart representation of an algorithm:



The corresponding code is given by in Listing 3.4.

```
#include <iostream>
using namespace std;

int main() {
    string name = "Alice";
    int age = 30;
    double height = 1.7; // In meters

    // Chaining << to output text and variables together
    cout << "Name:_" << name << ",_Age:_" << age << ",_Height:_"
        << height << "_meter" << endl;

    return 0;
}
```

Listing 3.4: Output using variables

In the example, given in Listing 3.4,

- we declare three variables: **name** (string), **age** (int), and **height** (double), assigning them values to represent a person's name, age, and height;
- we then use a single **cout** statement to output a sentence that includes both the literal text describing the data and the values of the variables themselves; and
- the **<<** operator is used to chain the outputs together, seamlessly integrating the variable values with the text.

Question 3.2

What is the output of the program given in Listing 3.4? Copy the program into your programming environment, run it and check whether the output you gave agrees with what the program gives.

3.3.7 Advanced Formatting

C++ supports formatting of various data types using the **iostream** and **iomanip** libraries. **setw** is used to set field width and **setprecision** for controlling precision of floating point numbers.

Run the example below and experiment with the formatting:

```
#include <iostream>
#include <iomanip> // Required for setw and setprecision
using namespace std;

int main() {
    // Integer formatting
    int myInt = 123;
    cout << "Integer_formatted_with_setw(10):_" << setw(10)
         << myInt << " " << endl;

    // String formatting
    string myString = "Hello";
    cout << "String_formatted_with_setw(20):_" << setw(20)
         << myString << " " << endl;

    // Char formatting
    char myChar = 'A';
    cout << "Char_formatted_with_setw(5):_" << setw(5)
         << myChar << " " << endl;

    // Floating-point formatting
    double myDouble = 123.456789;
    cout << "Floating-point_without_formatting:_" << myDouble << endl;
    cout << "Floating-point_with_setprecision(5):_"
         << setprecision(5) << myDouble << endl;
```

```

cout << "Floating-point with setprecision(9):_"
      << setprecision(9) << myDouble << endl;
cout << "Fixed-floating-point with setprecision(3):_"
      << fixed << setprecision(3) << myDouble << endl;

// Resetting formatting
cout << resetiosflags(ios::fixed | ios::floatfield);
cout << "After resetting:_" << myDouble << endl;

return 0;
}

```

Listing 3.5: Format Output

In this example:

- `setw(10)`, `setw(20)`, and `setw(5)` are used to set the field width for an integer, string, and char, respectively. This determines the minimum number of characters to be used for output, padding with spaces if necessary.
- `setprecision(5)` and `setprecision(9)` control the number of significant digits for floating-point output. When used with `fixed`, such as `fixed` `||` `setprecision(3)`, it sets the number of digits after the decimal point instead.
- `resetiosflags(ios::fixed — ios::floatfield)` is used to reset the formatting effects of `fixed` and precision settings to their defaults, affecting subsequent output.

3.4 Operators

Operators in programming are symbols or tokens that represent specific actions or computations to be performed on one or more operands. These actions can range from simple arithmetic operations like addition and subtraction to more complex actions like comparison, assignment, and logical operations. Operators allow programmers to manipulate data, perform calculations, and control the flow of execution in a program.

Expressions in programming are combinations of operands (variables, constants, or literals) and operators that produce a single value. Expressions can be as simple as a single variable or value, or they can be more complex, involving multiple operands and operators.

Operators link to expressions by specifying how the operands should be manipulated or combined to produce a result. When an operator is applied to one or more operands within an expression, it performs the specified operation and produces a new value. This value can then be used as part of a larger expression or stored in a variable for later use.

Expressions, on the other hand, link to operators and operands by combining them in specific ways to produce a single value. An expression can consist of one or more operands, connected by one or more operators. The operators determine how the operands are combined or manipulated to produce the final result of the expression.

For example, consider the following expression:

```
int result = 5 + 3 * 2;
```

In this expression, the addition operator (+) and the multiplication operator (*) link to the operands (5, 3, and 2) to produce a final result. First, the multiplication operator (*) is evaluated, and $3 * 2$ yields 6. Then, the addition operator (+) is evaluated, and $5 + 6$ yields 11. Finally, the result 11 is assigned to the variable 'result'.

In summary, operators specify actions or computations to be performed on operands, expressions combine operands and operators to produce single values, and expressions link to operators and operands by specifying how they should be combined or manipulated.

3.4.1 Assignment

Before a variable may be used, a value **MUST** be assigned to the variable. The following program would give unpredictable results due to the variable **radius** being defined and not assigned an initial value.

```
#include <iostream>

using namespace std;

int main() {

    const int pi = 3.1415;

    int radius;

    cout << "The area of a circle with a radius of 10cm is : "
    << pi*radius*radius;
}
```

Listing 3.6: Unpredictable results

To assign values to variables the assignment operator (=) is used. The expression on the right-hand side of the assignment operator is evaluated and the result is assigned to the variable on the left-hand side of the operator. The resultant type of the expression must be the same as the type that the variable is defined as. For example, the following program assigns the value of 10 to the integer variable radius.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6
7     const int pi = 3.1415;
8
9     int radius;
10    radius = 10;
11
12    cout << "The area of a circle with a radius of 10cm is : "
13    << pi*radius*radius;
```

Listing 3.7: Assigning a value to radius

Note that the variable `radius` is defined as a place-holder for an Integer value and therefore can take on values from -2147483648 to 2147483647 inclusive. Variables are not at all useful unless it is possible to retrieve the value that the variable represents and assign “user values” to the variables. We can write lines 9 and 10 in a single statement, that is: `int radius = 10;`

It is always good coding practice to assign default values to variables when they are first created. This will ensure that when the code executes, at least the variable has a valid value. Note, modern C++ helps with this. The compiler in many instances assigns a default value to a variable when it is created. An `int` value will default to 0. A variable having a valid value does not mean that your code is error free. It is still important to check the value, especially when you want to, for example divide by the radius. Division by zero will result in a logical error. Why you ask, because the compiler does not know what to do when it occurs. Is the answer infinite, undefined or what?

3.4.2 Mathematical

Mathematical operators are classified as either unary or binary. The addition (+) and subtraction (-) operators are both unary and binary, while the other operators are all binary. Mathematical operators in C++ perform arithmetic operation on operands (values, variables and constants). These operators allow you perform calculations and solve mathematical problems. The mathematical operators are: addition(+), subtraction (-), multiplication (*), division (/), modulus (%), increment (++) and decrement (--), and unary plus (+) and unary minus (-). A brief description and example of each of these operators is given in the list below.

- **Addition (+):** Determines the sum of two operands.

```
int result = 5 + 3; // result is 8
```

- **Subtraction (-):** Calculates the difference of two operands by subtracting the second operand from the first operand.

```
int result = 10 - 5; // result is 5
```

- **Multiplication (*):** Determines the product of the two operands.

```
int result = 4 * 3; // result is 12
```

- **Division (/):** Divides the left operand by the right operand. If both operands are integers, integer division is performed, and any fractional part is truncated. If one of the operands is a floating point value, floating point division is performed and the result is a floating point value.

```
double result_double = 10.0 / 3.0; // result is 3.33333
double result_int = 10 / 3; // result is 3
```

- **Modulus (%)**: Determines the remainder when dividing the left operand by the right operand.

```
int result = 10 % 3; // result is 1
```

- **Increment (++) and Decrement (--)**: ++ increases the value of its operand by 1, while -- decreases the value of its operand by 1. There are prefix and postfix versions of these operands as well.

```
int i = 5;
i++; // i is now 6 (postfix)
++i; // i is now 7 (prefix)
```

- **Unary Plus (+) and Unary Minus (-)**: The unary plus operator (+) indicates the positive value of its operand, while the unary minus operator (-) indicates the negative value of its operand.

```
int positive = +5; // positive is 5
int negative = -5; // negative is -5
```

These mathematical operators can be combined and used in expressions to perform complex calculations. Understanding how to use these operators effectively is essential for solving mathematical problems and performing numerical computations in C++ programs.

3.4.2.1 Compound Assignment Operators

Compound assignment operators provide a shorthand way to update the value of a variable based on its current value and another value. Since the variable's own value is used in the assignment it is important that the variable being updated already has a value.

Examples:

- **+=** (addition assignment): **a += b** is equivalent to **a = a + b**
- **-=** (subtraction assignment): **a -= b** is equivalent to **a = a - b**

There are compound assignment operators available for arithmetic, bitwise and shift operations.

3.4.3 Comparison operators

In C++, comparison operators are used to compare the values of two operands and determine their relationship. These operators return a boolean value (**true** or **false**) based on whether the comparison is true or false. Here's an explanation of the comparison operators in C++ along with examples:

- **Equal to (==)**: This operator checks if the values of two operands are equal. If the values are equal, it returns **true**; otherwise, it returns **false**.

```

int a = 5;
int b = 7;
bool result = (a == b); // result is false

```

- **Not equal to (!=):** This operator checks if the values of two operands are not equal. If the values are not equal, it returns **true**; otherwise, it returns **false**.

```

int x = 10;
int y = 10;
bool result = (x != y); // result is false

```

- **Greater than (>):** This operator checks if the value of the left operand is greater than the value of the right operand. If the condition is true, it returns **true**; otherwise, it returns **false**.

```

int p = 15;
int q = 10;
bool result = (p > q); // result is true

```

- **Less than (<):** This operator checks if the value of the left operand is less than the value of the right operand. If the condition is true, it returns **true**; otherwise, it returns **false**.

```

int m = 7;
int n = 12;
bool result = (m < n); // result is true

```

- **Greater than or equal to (>=):** This operator checks if the value of the left operand is greater than or equal to the value of the right operand. If the condition is true, it returns **true**; otherwise, it returns **false**.

```

int x = 20;
int y = 20;
bool result = (x >= y); // result is true

```

- **Less than or equal to (<=):** This operator checks if the value of the left operand is less than or equal to the value of the right operand. If the condition is true, it returns **true**; otherwise, it returns **false**.

```

int a = 8;
int b = 10;
bool result = (a <= b); // result is true

```

These comparison operators are commonly used in conditional statements, such as **if** statements and loops, to control the flow of program execution based on the evaluation of conditions. Understanding and mastering these operators is essential for writing effective and efficient C++ code.

3.4.4 Boolean Expressions

A boolean expression (condition) is any number of expressions that either results in a **true** or **false**. For example, the expression that consists of a single condition, `2 > 3`, results in **false**. The following C++ condition will either in **true** or **false**, depending on the value of the variable used in the expression:

```
(SavingsAccountBalance > 1000000.00).
```

In C++, the boolean value **true** is represented by the integer value 1. This convention allows boolean expressions to be evaluated in contexts where integer values are expected, such as in conditional statements ('if', 'while', etc.) and logical operators (&&, ||, etc.). When a boolean expression evaluates to **true**, it is internally represented as the integer value 1. Similarly, the boolean value **false** is represented by the integer value 0. This allows for seamless integration of boolean logic with arithmetic and bitwise (& and |) operations in C++.

Conditions that comprise more than one expression are joined by the binary logical (Boolean) operators || (or) and && (and). The opposite of a condition (or expression) is determined by using the unary logical (Boolean) operator ! (not). Examples here are:

```
(age == 18) || (age == 19) || (age == 20)

(age > 17) && (age < 21)

!(dadsBankBalance > 10000.00)

!(loanApproved)

(age >= 18) && !(age >= 21)    // what does this mean?

!(age <= 17) && !(age >= 21)  // what does this mean?
```

You might have noticed that the above examples of Boolean expressions are grouped using brackets. This is normally the case when using expressions like these.

At a first glance, the operators <= and >= might seem strange to you. They just mean “is smaller than or equal to” and “is greater than or equal to”, respectively.

The following truth tables illustrate how the logical operators are used in Boolean expressions. **A** and **B** represent logical expressions like the ones given here above. The **not** (!) operator returns the opposite value of the expression:

A	!A
true	false
false	true

The **and** (&&) operator applied on two conditions (e.g. **A** && **B**) will yield **true** only if both conditions are **true**. The expression **A** && **B** will be **false** if at least one of, or both A and B are **false**.

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

The **or** (**||**) operator applied on two conditions (e.g. **A || B**) will yield **true** if at least one, or both conditions are **true**. The expression **A || B** will only be **FALSE** if both A and B are **FALSE**.

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

The syntax for the C++ logical operators is as follows:

- Not: `!<booleanExpression>`
- And: `<booleanExpression1> && <booleanExpression2>`
- Or: `<booleanExpression1> || <booleanExpression2>`

The **!**, **&&** and **||** boolean operators may be mixed. Precedence of the operators are as follows: Not (**!**) has higher precedence than **&&**, which in turn has a higher precedence than **||**.

Question 3.3

Assuming that the variables **x**, **y** and **z** are defined and have valid integer values. What do each of the following expressions test?

- `(x>0) && (x<10)`
- `(y<0) || (y>50)`
- `(x>y) && (y>z)`
- `(y=20) || (y>x) && (y<z)`
- `(y=20) && (y>x) && (y<z)`

Question 3.4

Assuming that the variables **x**, **y** and **z** are defined and have valid boolean values. Set up a truth table for each of the expressions.

- `!(x && y)`
- `!x && y || z`

- `!x && !y`
- `!x || !y`

Question 3.5

If `gender` has the value 'F', `age` has the value 19 and `Hobby` has the value 'Playing saxophone',

- how will the following Boolean expression be evaluated:
`(gender == 'F') || (age > 15) && (hobby == 'Stamps')`
- how will this evaluation change when the expression has an extra pair of brackets:
`((gender == 'F') || (age > 15)) && (hobby == 'Stamps')`

3.4.4.1 Short-Circuiting Operators

Short-circuiting is used to avoid unnecessary calculation or evaluation of expressions. The expression is evaluated from left to right. Under certain cases the expression will be evaluated before all conditions have been individually evaluated.

Short-circuiting occurs when evaluating `&&` (AND) and `||` (OR) operators. When the left-hand side of the `&&` (AND) operator is false, the expression will always be false, so checking the right-hand side is unnecessary.

Similarly, when the left-hand side of the `||` (OR) operator is true, the expression will always evaluate to true, regardless of the right-hand side of the expressions.

```
#include <iostream>
using namespace std;

bool isPositive(int number) {
    cout << "Checking_if_positive..." << endl;
    return number > 0;
}

bool isEven(int number) {
    cout << "Checking_if_even..." << endl;
    return number % 2 == 0;
}

int main() {
    int num = -2;
    if (isPositive(num) && isEven(num)) {
        cout << "Number_is_both_positive_and_even." << endl;
    } else {
        cout << "Number_is_not_both_positive_and_even." << endl;
    }

    int num = 3;
    if (isPositive(num) && isEven(num)) {
```

```

        cout << "Number is both positive and even." << endl;
    } else {
        cout << "Number is not both positive and even." << endl;
    }

    int num = 2;
    if (isPositive(num) && isEven(num)) {
        cout << "Number is both positive and even." << endl;
    } else {
        cout << "Number is not both positive and even." << endl;
    }
    return 0;
}

```

Listing 3.8: Short-circuit operations

In the example Listing 3.8 we want to check whether a number is both positive and even. If the `isPositive(num)` returns `false` then the `isEven(num)` function will not be called because the overall condition (positive and even) cannot be true even if the number is even. Thus, the evaluation of `isEven(num)` is skipped.

3.4.5 Operator Precedence

Operator precedence in C++ determines the order in which operators are evaluated in an expression. It defines which operators are evaluated first and which are evaluated later when an expression contains multiple operators. Understanding operator precedence is crucial for writing expressions that yield the expected results and avoiding ambiguity.

All mathematical expressions are evaluated according to the precedence of the operator involved. For example the expression:

$2 + 10 * 2$

will result in the answer of 22 and not 24. This is because multiplication (*) has higher precedence than addition (+). Brackets can change the order in which expressions are evaluated. For example, adding brackets to the expression given above as follows:

$(2 + 10) * 2$

will ensure that 2+10 is worked out first before the answer is multiplied by 2, this will give a final answer of 24.

The following list shows the precedence of the operators. /, *, %, +, -

/, * and % have higher priority than + and -. The priority on a given level is equal, that is + and - have the same priority. This means that if an expression only has operators of the same precedence, the expression will be evaluated from left to right, for example:

$3 * 6 / 2$

will result in 9 ($3*6 = 18$, $18 / 2 = 9$ from left to right).

The precedence table for the use of Boolean and mathematical expressions is given in Table 3.2. The operators at the top of the table have highest priority and are therefore applied first.

	Operator	Associativity
1	!, unary -, unary +	←
2	/, *, %	→
3	+, -,	→
4	<, <=, >, >=	→
5	==, !=	→
6	&&	→
7		→
8	=, +=	←

Table 3.2: Operator Precedence

Understanding operator precedence helps in writing clear and concise expressions without relying heavily on parentheses. It's essential to be aware of the precedence rules to avoid unintended behavior and ensure the correctness of your code.

3.5 Input

To place values into variables, as described in Section 3.4.1, does not solve the problem of accepting different values user input into variables. The function `cin` is used to read values from the console (or the keyboard). The operation `>>` is defined to stream input from the keyboard into the variables. `cin` is defined in the library `iostream` and therefore, as with `cout`, it is necessary to specify the library using the preprocessor directive. Listing 3.9 is an example program that reads an integer value from the keyboard and then uses this value in the subsequent output statement. Note the use of scoping operator to gain access to the `std` namespace.

```
#include <iostream>

int main() {
    int x;

    std::cout << "Type in a value: ";
    std::cin >> x;

    std::cout << "x squared is: " << x * x << std::endl;

    return 0;
}
```

Listing 3.9: Receiving input

Question 3.6

What output do you expect from the program? Run the program. Is the output the same as you expected?

Question 3.7

Type in the following program:

```

#include <iostream>

int main() {
    float x, y;

    std::cout << "Type in two values: ";
    std::cin >> x >> y;

    std::cout << "x times y is: " << x * y << std::endl;

    return 0;
}

```

Listing 3.10: Typing in two values

If the values of 10.6 and 30.000524 are typed in for x and y respectively, what is the output of the program?

Question 3.8

If the variables being read in Listing 3.10 with the `cin` statement were read in using two statements, that is:

```

std::cin >> x;
std::cin >> y;

```

would the manner in which the values are read in change?

Question 3.9

Run the program, as changed in Question 8, and type two values in with a space between the values before pressing [Enter]. For example, type in:

10 50.3

and then press [Enter].

Variables may also be used to read in values of type string. To be able to manipulate strings in your program, you need to include the `string` library.

Consider the following program.

```

#include <iostream>
#include <string>

int main() {
    std::string surname;
    char initial;

    std::cout << "Type in your surname: ";
    std::cin >> surname;

    std::cout << "Type in one of your first initial: ";
    std::cin >> initial;

    std::cout << "Hello " << initial << " " << surname << std::endl;
}

```

```

    return 0;
}

```

When you run the program and enter “Sithole” for the surname and ‘V’ as the initial, the program will print: Hello V Sithole. What happens if the surname is “Van der Merwe”? The output is:

```

Type in your surname: Van der Merwe
Type in one of your first initial: Hello d Van

```

The program took “Van” as the surname that was typed in and ‘d’ as the initial. To fix this you need to use a special function, `getline` that does not use space as a delimiter.

Replace `std::cin >> surname;` with `std::getline(std::cin, surname);` and rerun the program. Test the program with a surname of “Van der Merwe” and initial of ‘V’. Is the output what you expected?

Below is the updated program.

```

#include <iostream>
#include <string>

int main() {
    std::string surname;
    char initial;

    std::cout << "Type in your surname: ";
    std::getline(std::cin, surname);

    std::cout << "Type in one of your first initial: ";
    std::cin >> initial;

    std::cout << "Hello " << initial << " " << surname << std::endl;

    return 0;
}

```

Question 3.10

How many characters may a string comprise of?

The input statement consisting of the `getline` function can also be used by specifying the maximum number of character the string should comprise of. Assuming the surname should not exceed 15 characters, then the statement can be modified as follows:

```

std::getline(std::cin, surname, 15);

```

When you want to read a single character, `cin.get(char& <variable_name>)` can be used. `cin.get(char& var)` reads a single character from the input and stores it in the provided character variable. You can also use `cin.get(char* charArray, int n)` to read a stream of characters until the newline (`\n`) character or `n-1` characters are read. This method is useful for reading a line of text that includes spaces, and has a known maximum size.

Another method is using `cin.read(char* buffer, int n)`. `cin.read(char* buffer, int n)` is used to read a specific number of characters (`n`) from the input stream and store them into a buffer. This includes reading the newline (`\n`) character into the buffer. This method is useful for reading text that spans multiple lines and is of known size.

It is important to note that:

- When reading in values from the keyboard, one should always have accompanying explanatory `cout` statements before the `cin` statements. Then the user of the program will know what to enter, for example:

```
cout << "Enter two numbers separated by a space:" );
```

- When we discuss pointers and arrays, we will revisit string input.

3.6 Document Change Log

Date	Change
4 Mar 2024	Initial preparation and presentation of the document.